

our program, it will be incrementally rewritten to be a right-associated sequence of FlatMap constructors:

```
FlatMap(a1, a1 =>
  FlatMap(a2, a2 =>
    FlatMap(a3, a3 =>
      ...
      FlatMap(aN, aN => Return(aN))))
```

If we now pass our example program `p` to `run`, it'll continue running indefinitely without a stack overflow, which is what we want. Our `run` function won't overflow the stack, even for infinitely recursive IO programs.

What have we done here? When a program running on the JVM makes a function call, it'll push a frame onto the call stack in order to remember where to return after the call has finished so that the execution can continue. We've made this program control explicit in our IO data type. When `run` interprets an IO program, it'll determine whether the program is requesting to execute some effect with a `Suspend(s)`, or whether it wants to call a subroutine with `FlatMap(x, f)`. Instead of the program making use of the call stack, `run` will call `x()` and then continue by calling `f` on the result of that. And `f` will immediately return either a `Suspend`, a `FlatMap`, or a `Return`, transferring control to `run` again. Our IO program is therefore a kind of *coroutine*⁶ that executes cooperatively with `run`. It continually makes either `Suspend` or `FlatMap` requests, and every time it does so, it suspends its own execution and returns control to `run`. And it's actually `run` that drives the execution of the program forward, one such suspension at a time. A function like `run` is sometimes called a *trampoline*, and the overall technique of returning control to a single loop to eliminate the stack is called *trampolining*.

13.3.2 Trampolining: a general solution to stack overflow

Nothing says that the `resume` functions in our IO monad have to perform side effects. The IO type we have so far is in fact a general data structure for trampolining computations—even *pure* computations that don't do any I/O at all!

The `StackOverflowError` problem manifests itself in Scala wherever we have a composite function that consists of more function calls than there's space for on the call stack. This problem is easy to demonstrate:

```
scala> val f = (x: Int) => x
f: Int => Int = <function1>

scala> val g = List.fill(100000)(f).foldLeft(f)(_ compose _)
g: Int => Int = <function1>

scala> g(42)
java.lang.StackOverflowError
```

We construct a composite function `g` that consists of 100,000 functions where each one calls the next.



⁶ If you aren't familiar with the term *coroutine*, you may want to check out the Wikipedia page (<http://mng.bz/ALiI>), but it's not essential to following the rest of this chapter.

And it'll likely fail for much smaller compositions. Fortunately, we can solve this with our IO monad:

```
scala> val f: Int => IO[Int] = (x: Int) => Return(x)
f: Int => IO[Int] = <function1>

scala> val g = List.fill(100000)(f).foldLeft(f) {
  |   (a, b) => x => Suspend(() => a(x).flatMap(b))
  | }
g: Int => IO[Int] = <function1>

scala> val x1 = run(g(0))
x1: Int = 0

scala> val x2 = run(g(42))
x2: Int = 42
```

Create a large,
left-nested chain
of flatMap calls.



But there's no I/O going on here at all. So IO is a bit of a misnomer. It really gets that name from the fact that Suspend can contain a side-effecting function. But what we have is not really a monad for I/O—it's actually a monad for tail-call elimination! Let's change its name to reflect that:

```
sealed trait TailRec[A] {
  def flatMap[B](f: A => TailRec[B]): TailRec[B] =
    FlatMap(this, f)
  def map[B](f: A => B): TailRec[B] =
    flatMap(f andThen (Return(_)))
}
case class Return[A](a: A) extends TailRec[A]
case class Suspend[A](resume: () => A) extends TailRec[A]
case class FlatMap[A, B](sub: TailRec[A],
  k: A => TailRec[B]) extends TailRec[B]
```

We can use the TailRec data type to add trampolining to any function type $A \Rightarrow B$ by modifying the return type B to TailRec[B] instead. We just saw an example where we changed a program that used $\text{Int} \Rightarrow \text{Int}$ to use $\text{Int} \Rightarrow \text{TailRec}[\text{Int}]$. The program just had to be modified to use flatMap in function composition⁷ and to Suspend before every function call.

Using TailRec can be slower than direct function calls, but its advantage is that we gain predictable stack usage.⁸

⁷ This is just Kleisli composition from chapter 11. In other words, the trampolined function uses Kleisli composition in the TailRec monad instead of ordinary function composition.

⁸ When we use TailRec to implement tail calls that wouldn't be otherwise optimized, it's faster than using direct calls (not to mention stack-safe). It seems that the overhead of building and tearing down stack frames is greater than the overhead of having all calls be wrapped in a Suspend. There are variations on TailRec that we haven't investigated in detail—it isn't necessary to transfer control to the central loop after every function call, only periodically to avoid stack overflows. We can, for example, implement the same basic idea using exceptions. See Throw.scala in the chapter code.